

2025년 1학기 데이터통신 12주차

과제 보고서

-파일 공유-



202001156 정보통계학과 김수영

과제 해결

26	15.553708	192.168.42.130	192.168.42.1	TCP	74 [TCP Retransmission] 55056 + 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3708446801 TSecr=0 NS=128
27	15.580139	192.168.42.130	192.168.42.129	UDP	60 43060 + 5203 Len=14
28	15.580413	192.168.42.129	192.168.42.130	UDP	60 5203 + 43060 Len=13
29	16.001673	Vmware_c0:00:01	Vmware_c0:00:01	ARP	60 who has 192.168.42.1? Tell 192.168.42.130
30	16.001687	Vmware_c0:00:01	Vmware_26:8d:af	ARP	42 192.168.42.1 is at 00:50:56:c0:00:01
31	16.577657	192.168.42.130	192.168.42.1	TCP	74 [TCP Retransmission] 55056 + 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3708447825 TSecr=0 NS=128
32	16.964721	192.168.42.129	192.168.42.1	DNS	93 Standard query 0x3c8b A kr.archive.ubuntu.com.localdomain
33	16.964827	192.168.42.129	192.168.42.1	DNS	86 Standard query 0x2b3e AAAA ntp.ubuntu.com.localdomain
34	16.964885	192.168.42.129	192.168.42.1	DNS	86 Standard query 0xae1c A ntp.ubuntu.com.localdomain
35	16.964934	192.168.42.129	192.168.42.1	DNS	92 Standard query 0x3733 SRV _http._tcp.kr.archive.ubuntu.com
36	16.964977	192.168.42.129	192.168.42.1	DNS	74 Standard query 0xe2f6 A ntp.ubuntu.com
37	16.965012	192.168.42.129	192.168.42.1	DNS	74 Standard query 0x3080 AAAA ntp.ubuntu.com
38	16.965046	192.168.42.129	192.168.42.1	DNS	81 Standard query 0x559c A kr.archive.ubuntu.com
39	16.965079	192.168.42.129	192.168.42.1	DNS	81 Standard query 0x4ce1 AAAA kr.archive.ubuntu.com
40	16.965112	192.168.42.129	192.168.42.1	DNS	93 Standard query 0x1666 AAAA kr.archive.ubuntu.com.localdomain
41	17.001652	192.168.42.130	192.168.42.1	TCP	74 [TCP Retransmission] 55056 + 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3708448849 TSecr=0 NS=128
42	18.625716	192.168.42.130	192.168.42.1	TCP	74 [TCP Retransmission] 55056 + 53 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3708449873 TSecr=0 NS=128

Clone of DC25

```

jellyfish319@jellyfish319-server:/mnt/hgfs/test$ python3 client.py test.jpg download/download.jpg
1) File information request... from 192.168.42.129:5203
>> File Size: 52650 byte
2) File download request...
downloaded 52650/52650 byte
>> download complete
jellyfish319@jellyfish319-server:/mnt/hgfs/test$ python3 client.py testtt.jpg download/download.jpg
1) File information request... from 192.168.42.129:5203
>> 404 Not Found: No File in Server
jellyfish319@jellyfish319-server:/mnt/hgfs/test$ python3 client.py testtt.jpg download/download.jpg
1) File information request... from 192.168.42.129:5203
>> 404 Not Found: No File in Server
jellyfish319@jellyfish319-server:/mnt/hgfs/test$

```

DC25

```

jellyfish319@jellyfish319-server:/mnt/hgfs/test$ python3 server.py --port 5203
UDP server (port 5203, MTU 1500)
INFO request for 'test.jpg' from ('192.168.42.130', 42090)
DOWNLOAD request for 'test.jpg' from ('192.168.42.130', 42090)
INFO request for 'testtt.jpg' from ('192.168.42.130', 45580)
INFO request for 'testtt.jpg' from ('192.168.42.130', 43060)

```

Corruption

```

jellyfish319@jellyfish319-server:/mnt/hgfs/test$ python3 client.py test.jpg download/corrupt.jpg
1) File information request... from 192.168.42.129:5203
>> File Size: 52650 byte
2) File download request...
downloaded 46650/52650 byte>> Timeout while receiving data
>> download complete
jellyfish319@jellyfish319-server:/mnt/hgfs/test$ _

```

```

DOWNLOAD request for 'test.jpg' from ('192.168.42.130', 33630)
INFO request for 'test.jpg' from ('192.168.42.130', 39528)
DOWNLOAD request for 'test.jpg' from ('192.168.42.130', 39528)
INFO request for 'test.jpg' from ('192.168.42.130', 52306)
INFO request for 'test.jpg' from ('192.168.42.130', 40675)
DOWNLOAD request for 'test.jpg' from ('192.168.42.130', 40675)

```

3483	1343.617307	192.168.42.130	192.168.42.129	ICMP	590 Time-to-live exceeded (Fragment reassembly time exceeded)
3484	1343.617391	192.168.42.130	192.168.42.129	ICMP	590 Time-to-live exceeded (Fragment reassembly time exceeded)



original

corruption

코드 설명

```
1 import argparse
2 import socket
3 import os
4
5 parser = argparse.ArgumentParser(description="UDP 파일 서버")
6 parser.add_argument('--mtu', type=int, default=1500, help='전송 단위 크기 (바이트)') # 기본값
7 parser.add_argument('--port', type=int, default=3034, help='서버 바인드 포트')
8 FLAGS = parser.parse_args()
9
10 # 기본 디렉토리 설정
11 BASE_DIR = os.getcwd()
12 file_table = {}
13 for fname in os.listdir(BASE_DIR):
14     path = os.path.join(BASE_DIR, fname)
15     if os.path.isfile(path):
16         file_table[fname] = {
17             'path': path,
18             'size': os.path.getsize(path)
19         }
20
21 # INFO 처리
22 def handle_info(sock, addr, filename):
23     info = file_table.get(filename)
24     print(f"INFO request for '{filename}' from {addr}") # 파일 요청 확인인
25     if info: # 파일이 있을 경우 전송
26         sock.sendto(str(info['size']).encode('utf-8'), addr)
27     else:
28         sock.sendto(b'404 Not Found', addr) # 파일이 없을 경우 404에러 전송
29
30 def handle_download(sock, addr, filename):
31     info = file_table.get(filename)
32     print(f"DOWNLOAD request for '{filename}' from {addr}") # 파일 다운로드 요청 확인
33     with open(info['path'], 'rb') as f:
34         while True:
35             chunk = f.read(FLAGS.mtu)
36             if not chunk:
37                 break
38             sock.sendto(chunk, addr) # 1500 바이트 단위로 파일 전송
39
40 def main():
41     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # "DGRAM": Unknown word.
42     sock.bind(('', FLAGS.port))
43     print(f"UDP server (port {FLAGS.port}, MTU {FLAGS.mtu})")
44     try:
45         while True:
46             data, addr = sock.recvfrom(4096)
47             text = data.decode('utf-8', errors='ignore').strip()
48             if text.startswith('INFO '):
49                 _, filename = text.split(' ', 1)
50                 handle_info(sock, addr, filename)
51             elif text.startswith('DOWNLOAD '):
52                 _, filename = text.split(' ', 1)
53                 handle_download(sock, addr, filename)
54     except KeyboardInterrupt:
55         print("\nTerminate")
56     finally:
57         sock.close()
58
59 if __name__ == '__main__':
60     main()
```

클라이언트로부터 파일 요청을 받으면 해당 파일을 전송하고 없으면 404 Not Found 에러를 전송한다. INFO를 이렇게 처리하고 1500바이트 단위로 파일을 전송한다. 클라이언트는 실습코드를 그대로 인용하였다. 파일 다운로드 요청은 INFO로 파일의 존재를 확인한 후 바로 파일을 전송해준다.